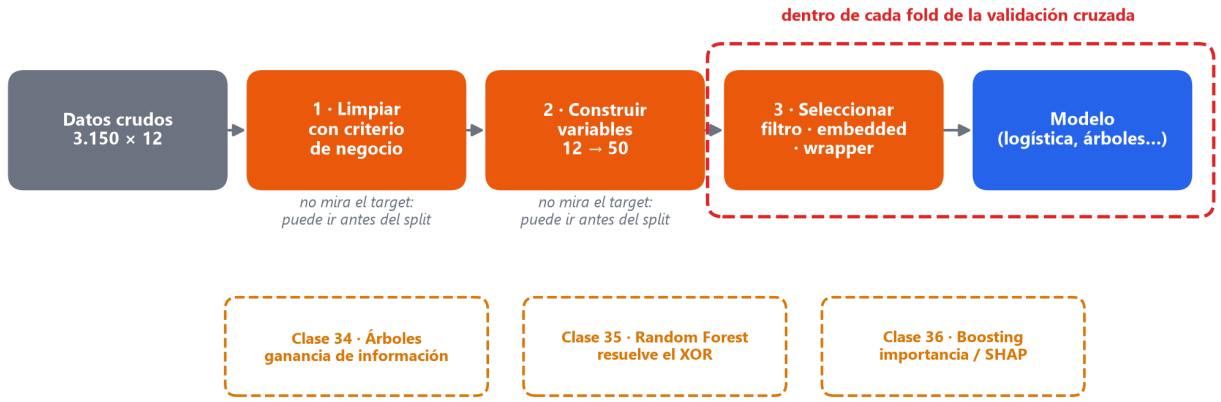
Clase 33 · Feature engineering y selección de variables

Profesor Eduardo Rodríguez · Etapa 7 del roadmap · dataset: fuga de clientes de una telco

La gran idea de la clase

Más variables no es un mejor modelo. El trabajo real, que según el profesor es el 70–80% de un proyecto, es **limpiar con criterio de negocio**, **construir variables con sentido** y **seleccionar sin hacer trampa**. Y la trampa más común es invisible: si una selección mira el target fuera de la validación cruzada, el AUC que reportas es inventado.

El flujo de trabajo



Lo que necesitas ANTES

- Regresión logística y lectura de AUC (Carpeta 2)
- Validación cruzada y data leakage del escalador (Clases 30–32)
- Correlación, ANOVA F y p-valor (Etapa 0 y Carpeta 1)
- pandas: groupby, describe, isna

Hacia dónde lleva

- **Clase 34 · árboles:** la ganancia de información es la información mutua de hoy
- **Clase 35 · Random Forest:** resuelve el XOR que la logística no ve
- **Clase 36 · Boosting:** importancia de variables y SHAP como selección embebida
- **Proyecto:** el checklist de esta clase aplica a cualquier modelo

Variable	Qué es	Variable	Qué es
numero_fallas	fallas de llamada	frec_sms	n° de SMS
reclamos	si reclamó (0/1)	n_contactos	n° de contactos distintos
antiguedad	meses como cliente	tipo_plan	1 o 2 (no se define)
monto	monto cobrado	estado	1 o 2 (no se define)
segundos	segundos de uso	edad	edad del cliente
frec_llamadas	n° de llamadas	clv	valor del cliente (customer lifetime value)
fuga	TARGET: 1 = el cliente se fue (15.7%)		

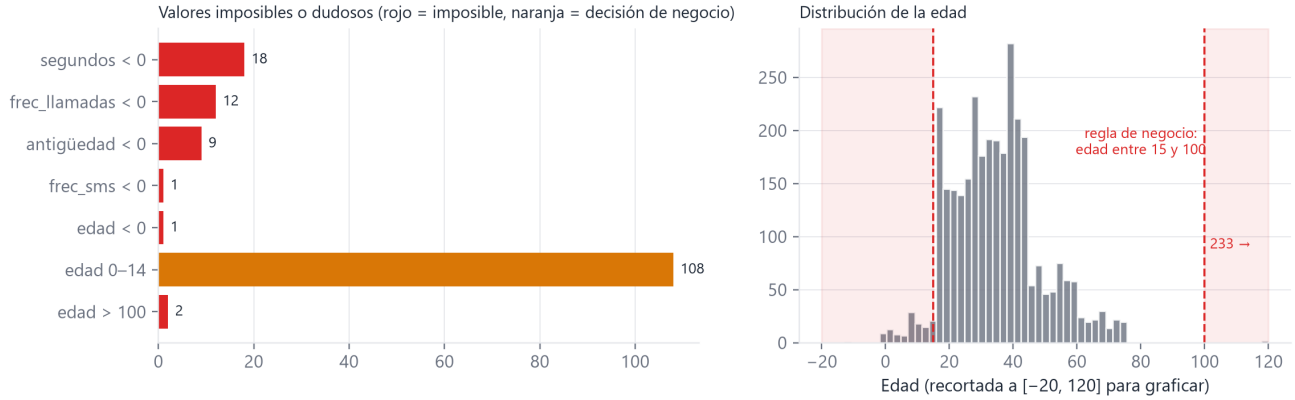
Clase 33 · Limpiar con criterio de negocio

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Ninguna técnica de selección arregla un dato imposible. La limpieza va antes, y el umbral lo pone el negocio, no la estadística.

Conceptos clave

1. **Valores imposibles.** Segundos negativos (mínimo −14.645), llamadas negativas, antigüedad −40, edades de −13 y 233 años. Los datos los traen a propósito.
2. **El umbral es de negocio.** «Edad entre 15 y 100 no sale de los datos: sale de saber que una telco no contrata a alguien de 233 años». Los 108 clientes de 0–14 años podrían ser cuentas a nombre de un menor: es una decisión, no un error.
3. **No borrar filas.** Es la advertencia más repetida del profesor: borrar una fila borra información de TODAS sus variables. Mejor: marcar como NaN, imputar o crear una categoría propia.
4. **Mediana, no media.** La mediana es robusta a los extremos. Un solo −14.645 arrastra la media de segundos; la mediana casi no se mueve.
5. **Revisa también.** Nulos (6 columnas con faltantes), 19 filas duplicadas y la ventana temporal de los datos antes de imputar mecánicamente.



Conteos reales de la base Churn. La barra naranja (edad 0–14) no es un error evidente: tratarla es una decisión de negocio.

Fórmulas y reglas

imposible → NaN → imputar con la mediana

edad válida: 15 ≤ edad ≤ 100

variables: solo información anterior al target

En Python

```
d.describe().T[['min', 'max']]
d.loc[d.segundos < 0, 'segundos'] = np.nan
d['edad'] = d.edad.where(
    d.edad.between(15, 100))
d.fillna(d.median(numeric_only=True))
```

Con los datos de la clase

- 3.150 clientes · 495 se fugan (15.71%)
- Imposibles: segundos 18 · llamadas 12 · antigüedad 9 · SMS 1 · edad 3
- Nulos: segundos 40 · llamadas 19 · reclamos 13 · antigüedad 5 · SMS 4 · contactos 2

Cuidado
Un «pájaro sobre el sensor» (la anécdota del profesor sobre Anglo American) genera outliers falsos; un fraude genera outliers reales. Antes de tocar un valor, pregunta de dónde viene.

Clase 33 · Construir variables: de 12 a 50

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Con 12 columnas no hay nada que seleccionar. El feature engineering fabrica candidatas con sentido de negocio, y con ellas aparece el problema real.

Conceptos clave

1. **Ratios.** Llamadas por mes, CLV por antigüedad, segundos por llamada… En la experiencia del profesor son «algo bien potente» y discriminan mejor que la variable cruda.
2. **Épsilon en el denominador.** x / (y + 1e-6): un solo infinito por dividir por cero rompe todo el pipeline que viene después.
3. **Otras familias.** Logaritmos (colas largas), indicadores binarios (tiene_reclamo, cliente_nuevo), cuartiles (clv_q1…q4), interacciones (reclamo × antigüedad) y one-hot (estado_2, plan_2).
4. **Redundancia por construcción.** reclamos y tiene_reclamo valen lo mismo (r = 1.000). Crear variables sin después seleccionar no mejora el modelo: lo infla.
5. **WOE (adelanto).** El profesor pide anotar el Weight of Evidence, usado en banca para discretizar antes de la logística. Se mencionó pero no se calculó en el notebook.

Fórmulas y reglas

ratio = numerador / (denominador + 1e-6)

log(x + 1) para variables con cola larga

13 ratios + 3 interacciones + 5 logs

+ 5 binarias + 12 cuartiles + 2 one-hot

+ 10 originales = 50 candidatas

En Python

```
eps = 1e-6
d['llamadas_por_mes'] = d.frec_llamadas /
(d.antiguedad + eps)
d['log_clv'] = np.log1p(d.clv)
d['clv_q'] = pd.qcut(d.clv, 4, labels=False)
pd.get_dummies(d, columns=['estado'],
drop_first=True)
```

Con los datos de la clase

- 12 originales → AUC 0.920 · 50 candidatas → AUC 0.941
- 11 pares con |r| > 0.90 entre las 50
- Pares idénticos: reclamos–tiene_reclamo, clv_alto–clv_q4

Cuidado
Construir variables no mira el target, así que puede ir antes del split. Pero ojo con la ventana temporal: una variable que usa información posterior al momento de predecir es leakage aunque no use la columna fuga.

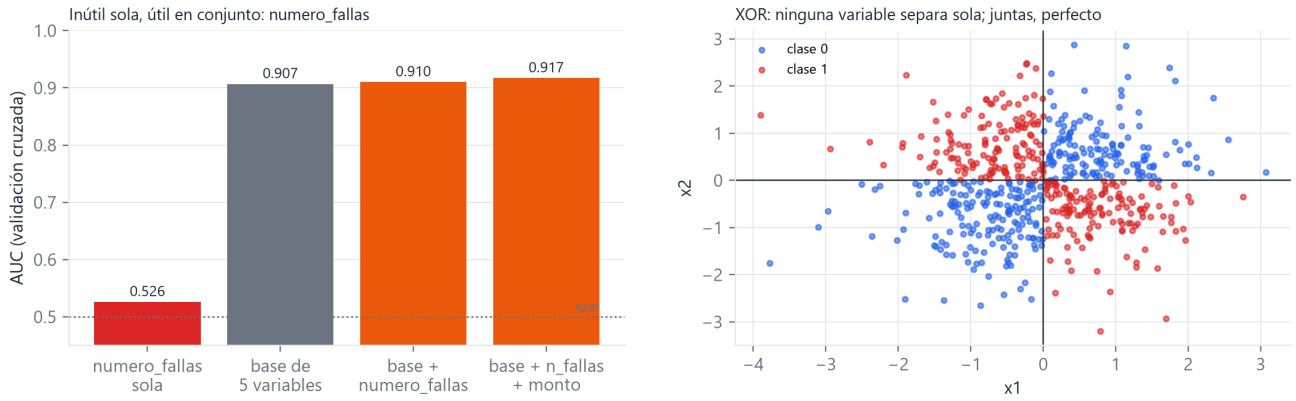
Clase 33 · Irrelevante, redundante o útil solo en conjunto

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Una variable puede sobrar por tres motivos distintos, y cada uno pide una decisión distinta. El más traicionero: la que es inútil sola, pero útil acompañada.

Conceptos clave

1. **Irrelevante.** No aporta nada, ni sola ni acompañada → se descarta.
2. **Redundante.** Repite la información de otra (r > 0.90) → se conserva una. Se desempata por información mutua y, a igualdad, por la más simple de mantener en producción, nunca por «la primera de la lista».
3. **Útil solo en conjunto.** numero_fallas tiene r = −0.009, información mutua 0 y p = 0.60. Sola da AUC 0.526 (azar), pero sumada a otras sube el modelo de 0.907 a 0.917.
4. **El caso XOR.** Dos variables que solas no separan nada (logística AUC 0.510) separan perfecto juntas: con la interacción x1·x2, o con un Random Forest, AUC = 1.000.
5. **La lección.** «Un p-valor alto no significa que la variable no sirve; significa que no sirve SOLA». Los filtros univariados matan estas variables.



Izquierda: AUC real con validación cruzada. Derecha: datos sintéticos XOR; ninguna recta horizontal ni vertical separa los colores.

Fórmulas y reglas

Redundancia: |r| > 0.90 → quedarse con una

Desempate: mayor información mutua

y a igualdad, la más simple de explicar

En Python

```
corr = X.corr().abs()
pares = corr.where(np.triu(
    np.ones(corr.shape), 1).astype(bool))
pares.stack().loc[lambda s: s > 0.9]
```

Con los datos de la clase

- Correlación con fuga: reclamos 0.51 · estado_2 0.32 · frec_llamadas −0.30
- numero_fallas: r −0.009 · p ANOVA 0.60
- XOR: logística 0.510 · + interacción 1.000 · RF 1.000

Cuidado

Un árbol poco profundo también falla en XOR: es voraz y evalúa un corte a la vez. Lo verás en la Clase 34.

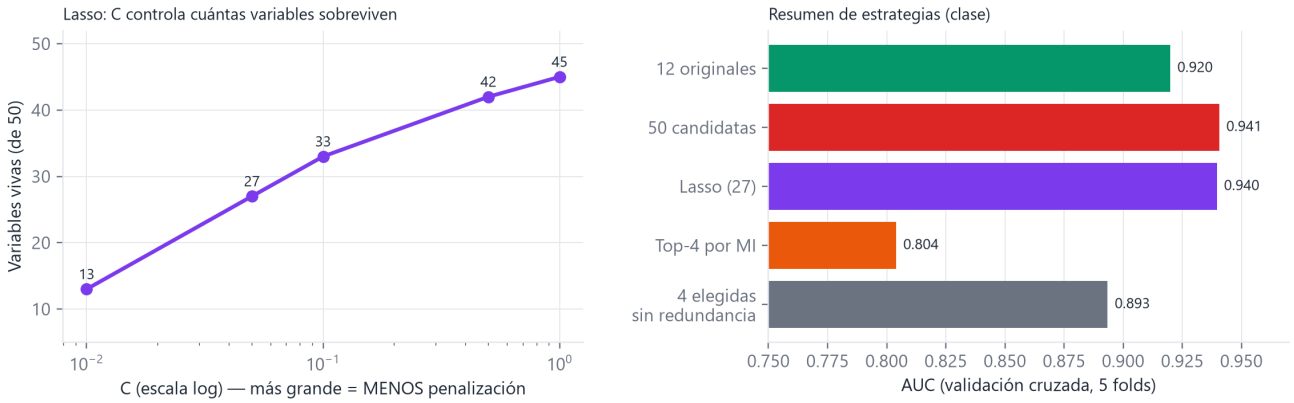
Clase 33 · Filter, wrapper y embedded

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Hay tres familias de selección. No compiten: son etapas de un embudo. Filtro para bajar de miles a cientos, embedded para bajar a decenas y wrapper solo si sobra cómputo.

Conceptos clave

1. **Filter.** Mide cada variable con una métrica, sin entrenar el modelo: varianza, correlación, información mutua (MI), ANOVA F, χ^2. Barato, pero no ve interacciones.
2. **Información mutua.** Detecta relaciones no lineales (es 0 solo si hay independencia). En sklearn se estima con vecinos: cambia con la semilla, así que fija random_state.
3. **Wrapper.** Entrena el modelo con distintos subconjuntos (RFE, forward, backward). Ve interacciones, pero es caro: con 50 variables hay $2^{50} \approx 10^{15}$ subconjuntos, unos 35.702 años a 1.000 modelos por segundo.
4. **Embedded: Lasso.** La penalización L1 lleva coeficientes exactamente a 0. En sklearn, C es el INVERSO de la penalización: C chico deja menos variables. Ridge (L2) no produce ceros.
5. **Top-k ingenuo.** Las 4 mejores por MI dan AUC 0.804 porque «gastan» dos cupos en segundos y log_segundos. Cuatro elegidas sin redundancia dan 0.893. Con k ≥ 6 la ventaja se diluye.



Izquierda: variables que sobreviven en el Lasso según C. Derecha: AUC con validación cruzada de cada estrategia en la clase.

Fórmulas y reglas

Var(binaria) = $\pi(1 - \pi)$

I(X; Y) ≥ 0 · = 0 solo si son independientes

F = CM entre / CM dentro

n° de subconjuntos = $2^p - 1$

Lasso: C chico → más coeficientes en 0

En Python

```
VarianceThreshold(threshold=0.01)
mutual_info_classif(X, y, random_state=0)
f_classif(X, y)
RFECV(LogisticRegression(), scoring='roc_auc')
LogisticRegression(penalty='l1',
    solver='liblinear', C=0.05)
```

Con los datos de la clase

- VarianceThreshold: 0 descartadas
- RFECV elige 42 de 50 · Lasso (C = 0.05) deja 27
- C: 0.01 → 13 · 0.1 → 33 · 1.0 → 45 variables
- Lasso es inestable: en 5 particiones solo 16 variables aparecen siempre

Cuidado

Escala las variables (StandardScaler) antes de RFE o Lasso: si no, el tamaño del coeficiente depende de las unidades y no de la importancia.

El Lasso requiere solver='liblinear' o 'saga'; el solver por defecto no admite L1.

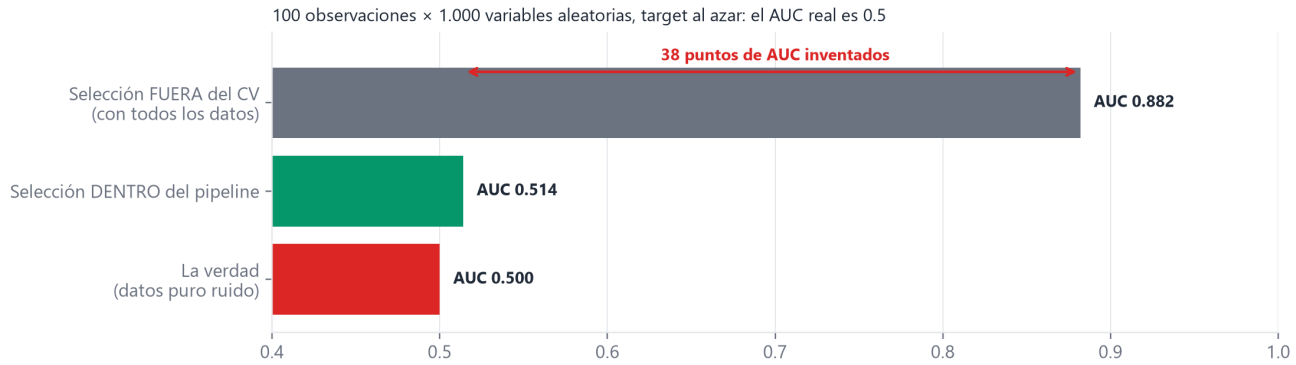
Clase 33 · La trampa: dónde vive la selección

Fuente: transcripción de la clase + diapositivas + notebook Python del profesor

Todo lo que mira el target (filtros, desempates, RFE, Lasso) debe hacerse DENTRO de cada fold de la validación cruzada. Si no, el modelo no generaliza: recuerda.

Conceptos clave

1. **El experimento.** 100 clientes × 1.000 variables de puro ruido, con un target al azar. El AUC verdadero es 0.5.
2. **Selección fuera del CV.** Elegir las 20 mejores con todos los datos y después validar da AUC 0.882: 38 puntos inventados de la nada.
3. **Selección dentro del pipeline.** Con SelectKBest dentro de un Pipeline pasado a cross_val_score, el AUC es 0.514: la verdad.
4. **Por qué pasa.** Con 1.000 variables y α = 0.05 se esperan α·p = 50 falsos positivos. El fold de validación contiene justo los casos que hicieron parecer buenas a esas variables.
5. **Extiende la regla de la Clase 32.** Ya sabías que el escalador se ajusta solo con train. La misma regla aplica a cualquier decisión que use el target o estadísticas del conjunto (media, mediana).



Resultado real del notebook de la clase. La misma métrica, los mismos datos: lo único que cambia es dónde vive la selección.

Fórmulas y reglas

Falsos positivos esperados = α · p

ANTES del split: limpieza, ratios, logs, one-hot

DENTRO del fold: imputar, escalar, filtrar, desempatar, RFE, Lasso

En Python

```
pipe = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('esc', StandardScaler()),
    ('sel', SelectKBest(mutual_info_classif,
k=12)),
    ('log',
LogisticRegression(max_iter=1000))])
cross_val_score(pipe, X, y, cv=skf,
scoring='roc_auc')
```

Con los datos de la clase

- Ruido: fuera del CV 0.882 · dentro 0.514 · verdad 0.500
- Pipeline honesto con los datos reales: AUC medio ≈ 0.90
- Por fold: entre 0.887 y 0.923

Cuidado
El profesor advierte: en la industria, un AUC de 0.84–0.85 en fuga ya es «extremadamente bueno». Un 0.90+ «de libro» suele esconder leakage. Desconfía de las métricas demasiado buenas.

Chuleta de Python y checklist del proyecto

Lo que vas a usar en cualquier modelo de aquí en adelante

Quiero…	En Python (scikit-learn)
Quitar variables casi constantes	VarianceThreshold(threshold=0.01)
Ranking por información mutua	mutual_info_classif(X, y, random_state=0)
Ranking por ANOVA F	f_classif(X, y) → (F, p)
Quedarme con las k mejores	SelectKBest(score_func, k=12) (dentro del Pipeline)
Eliminación recursiva con CV	RFECV(estimador, step=1, cv=5, scoring='roc_auc')
Lasso (selección embebida)	LogisticRegression(penalty='l1', solver='liblinear', C=…)
Imputar dentro del fold	SimpleImputer(strategy='median')
Encadenar pasos sin leakage	Pipeline([...]) + cross_val_score(pipe, X, y, cv=skf)
Folds que respetan el 15.7% de fuga	StratifiedKFold(5, shuffle=True, random_state=42)

Checklist metodológico

	Paso	Cómo sé que lo hice bien
☐	Limpia con criterio de negocio	umbrales que explicarías a un gerente, no solo a un estadístico
☐	Separa el test ANTES de todo	el test no se toca hasta el final
☐	Define la ventana temporal	las variables vienen de antes del período del target («junio hacia atrás»)
☐	Construye variables con sentido	ratios, logs, binarias; con épsilon en los denominadores
☐	Mide la redundancia	pares con |r| > 0.90; desempata por MI y simplicidad
☐	No mates variables por un p-valor alto	pueden servir en conjunto
☐	Mete la selección en el Pipeline	imputar, escalar, filtrar, RFE y Lasso dentro del fold
☐	Usa AUC, no exactitud	con 15.7% de fuga, decir «nadie se va» acierta 84.3%
☐	Desconfía de métricas perfectas	un AUC demasiado alto suele ser leakage
☐	Documenta por qué quedó cada variable	es lo que más modelos salva en producción

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales, preguntas clave y cuándo pasar a la Clase 34

Alertas del material

Diapositivas y notebook no siempre coinciden

- **Lasso:** la diapositiva 25 dice «25 de 50 sobreviven» y la 26 dice «0.938 con 25 variables». El notebook ejecutado da **27 de 50** y **AUC 0.9396**. Lo reproduje: el valor correcto es el del notebook.
- **Ranking de información mutua (diapositiva 22):** difiere levemente del notebook. No es un error: la MI se estima con vecinos y cambia con la semilla y con el orden de las columnas.
- **«6 vs 6» (diapositiva 23):** dice top-6 univariado AUC 0.8105 vs 6 elegidas a mano 0.9041. El notebook no tiene esa celda: su top-6 por MI da **0.897** (no 0.81), y su selección sin redundancia automática da 0.888. Toma la conclusión de la diapositiva con cautela: con 6 variables la ventaja de evitar redundancia ya no es clara.
- **Correlación segundos–frec_llamadas:** 0.907 en la diapositiva y 0.935 en el notebook. No es un error: 0.907 es con los datos crudos y 0.935 después de limpiar.

Otras observaciones

- El notebook de la clase carga `churn_telco.csv`, que no está en la carpeta. Los datos son los mismos del Excel `Base de datos Churn.xlsx`; tu notebook guiado ya lee el Excel.
- **tipo_plan** y **estado** nunca se definen (valen 1 o 2). Es justo la falta de diccionario de datos que el profesor critica.
- WOE / IV y KS / Gini se mencionan, pero no se calculan en esta clase. Están como adelanto en los apuntes.
- La transcripción dice «ocho tipos de técnicas» en una frase confusa sobre redes neuronales. No corresponde a ninguna lista de la clase: la taxonomía real es filter / wrapper / embedded.

Preguntas de repaso

#	Pregunta	Respuesta corta
1	¿Por qué la limpieza va antes de la selección?	Ninguna técnica corrige un dato imposible, y el criterio para limpiar es de negocio.
2	numero_fallas tiene p = 0.60. ¿La descartas?	No sin probarla en conjunto: sola da AUC 0.53, pero sube el modelo de 0.907 a 0.917.
3	¿Qué diferencia filter, wrapper y embedded?	Filter mide sin entrenar; wrapper entrena con subconjuntos; embedded selecciona mientras entrena (Lasso).
4	¿Por qué el top-4 por MI rinde peor que 4 elegidas a mano?	Gasta cupos en variables redundantes (segundos y log_segundos) y deja fuera reclamos.
5	¿Por qué seleccionar fuera del CV infla el AUC?	Los casos de validación ya influyeron en qué variables se eligieron: el modelo recuerda, no generaliza.
6	En sklearn, ¿C chico en Lasso deja más o menos variables?	Menos: C es el inverso de la penalización.

Criterio de dominio para pasar a la Clase 34

- ☐ Detecto valores imposibles y justifico la regla de negocio que aplico
- ☐ Construyo ratios con épsilon y sé qué familias de variables existen
- ☐ Distingo irrelevante, redundante y útil en conjunto, con un ejemplo de cada una
- ☐ Explico filter, wrapper y embedded, y cuándo usar cada uno
- ☐ Armo un Pipeline donde toda la selección vive dentro del fold
- ☐ Resuelvo los bloques 1 a 8 del cuaderno con al menos 80% de aciertos